

AquaLLM: Adaptive LLM Aquaculture Manager

PHU "LEON" HOANG

Introduction – A Virtual Pond

- **The Environment:** A simulated aquaculture pond for Largemouth Bass.
- **Dynamics:** Cyclic interactions between fish, food, probiotics, oxygen, NH3, disease, and parasites.
- **Goal:** Iterative policy improvement using LLM reasoning and simulation feedback, replacing original Evolutionary Algorithm.

OPTIMIZATION METRIC (THE GOAL)

$$\text{Fitness} = 0.55 \times \text{Yield} + 0.33 \times \text{SavingRate} + 0.12 \times \text{Healthiness}$$

The LLM agent must balance these three competing objectives.

LIVE SIMULATION: <https://jam.dev/c/6d18c995-6f4b-4d9a-9e1d-28bdda3d040b>

Largemouth Bass Aquaculture - 3D Visualization

Play

Restart

Loop: Off

Speed:

-

4

+

Jump:

<

>

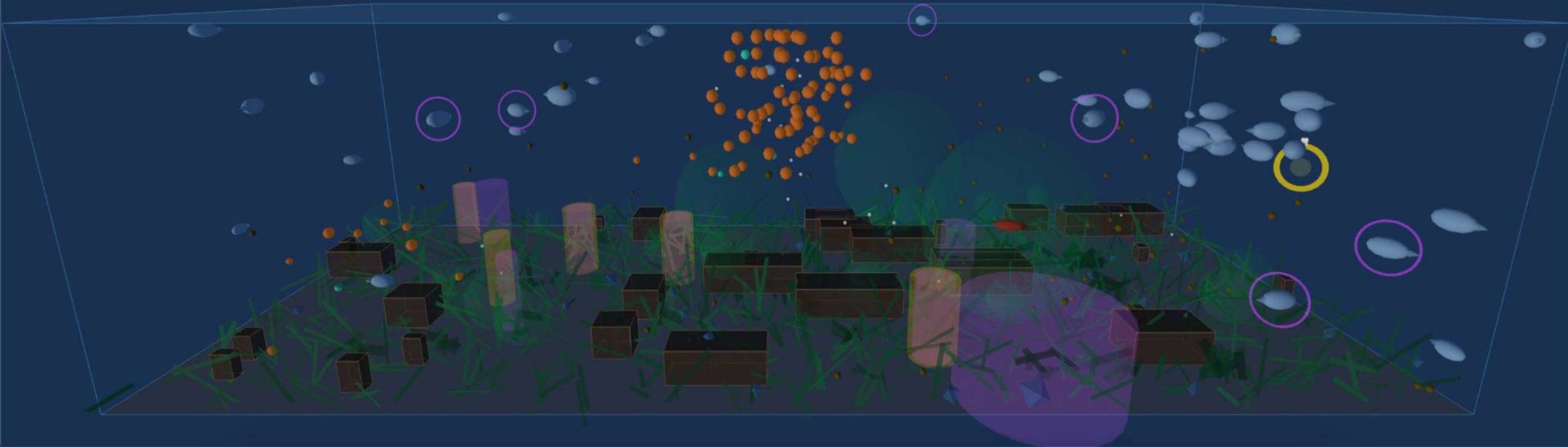
Day 24 | Hour 08 | Frame 0585/0720

Survival **91.1% (41/45)** Health **80.2** Fullness **40.2** Immunity **84.5** O2 **66.7** Infected **0** Parasitized **6** Deaths **2** Cannibalized **2** Objects **241** Hazards **32**

Fitness: 0.7395 Yield: 0.7347 Survival: 80.0% Cost: \$153.00 Saving: \$347.00 Fish: 45 Food: 15x / 4h @ Random Prob: 1x / 48h @ Top-Center O2: 1h / 20h @ Top-Center

 Fish  Food  Probiotic  O2  Fecal  Dead Fish  Pollutant  NH3  Disease  Parasite  Obstacle  Infected  Parasitized  Death  Cannibalized

Alive: 41 / 45



Previous Work

Generative Agents (Park et al.)

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. In Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23). <https://arxiv.org/abs/2304.03442>

Demonstrated that LLMs can simulate complex, interactive behaviors using memory streams and reflection.

→ AquaLLM Application:

We adapt this memory-driven reflection to environmental management, allowing our agent to "remember" past simulation days and adjust policies.

Voyager (Wang et al.)

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint arXiv:2305.16291. <https://arxiv.org/abs/2305.16291>

An embodied LLM agent in Minecraft that writes code and iteratively improves its skills without gradient training.

→ AquaLLM Application:

Similar to Voyager, AquaLLM uses tools (budget calculators) and environmental feedback to iteratively optimize its control policy.

Technical Approach – Modular Architecture

We use 4 toggleable cognitive modules to test different configurations to measure the impact of each module.



Observation (Obs)

Reads pond state (water quality, health, yield) in natural language.



Tool (Tool)

Executes actions and calculates critical budget constraints.



Planning (Plan)

Generates multi-step strategies projected over the growth cycle.

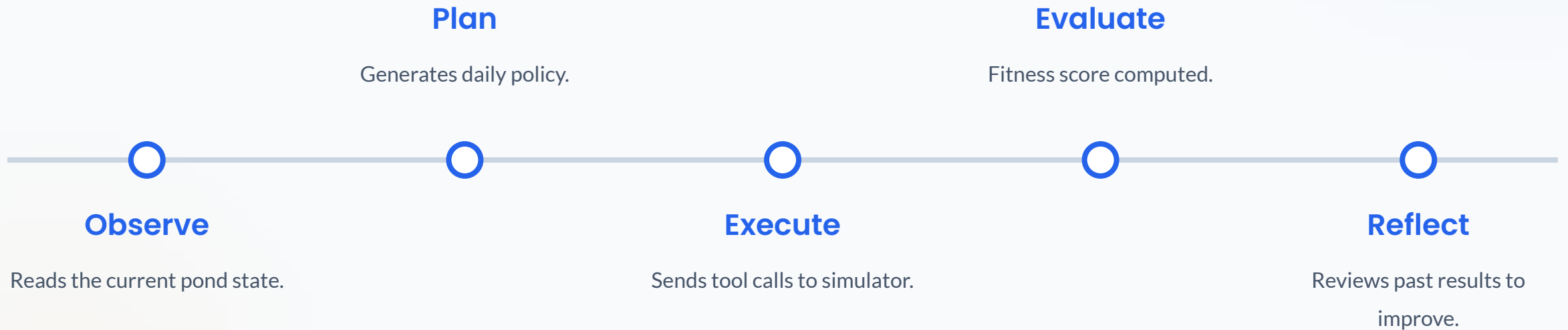


Memory (Mem)

Retains empirical results from previous runs to inform adjustments.

Implementation – The Iterative Optimization Loop

Model: Qwen2.5:7b (Local Ollama)



Data & Prompt

```
_CORE = (  
    "You manage an aquaculture fish pond (raising fish to keep alive, not catching). "  
    "{days}d ${budget} budget.\n"  
    "Fitness = 0.55*Yield + 0.33*Saving + 0.12*Health (Saving is 3x more important than Health!)\n"  
    "Yield=alive/max. Saving=(500-cost)/500. Over budget or all dead=Fitness 0.\n"  
    "CRITICAL: Spending less money DIRECTLY increases your Fitness. "  
    "Every $16.67 saved = +0.01 Fitness. Prefer higher intervals and lower quantities.\n"  
    "All policy values are integers. Focus on what to do next, not the past.\n"  
    "Locations: 0=Center 1=TL 2=TR 3=BL 4=BR 5=TC 6=BC 7=LC 8=RC 9=Random(best)\n"  
).format(days=AQUACULTURE_DAYS, budget=int(MAX_BUDGET))
```

```
_PLANNING_BLOCK = (  
    "Think step-by-step before outputting JSON.\n"  
    "1. Read stats. 2. Identify weakest. 3. Pick fix. 4. Check budget. 5. Output JSON.\n"  
)
```

```
_TOOLS_BLOCK = (  
    "Cost/day: food=(24/intv)*qty*$0.05 prob=(24/intv)*qty*$0.75 O2=(24/intv)*dur*$2.50. "  
    "Target: ${daily:.2f}/day. Use calculate_budget tool to verify.\n"  
).format(daily=_DAILY_BUDGET)
```

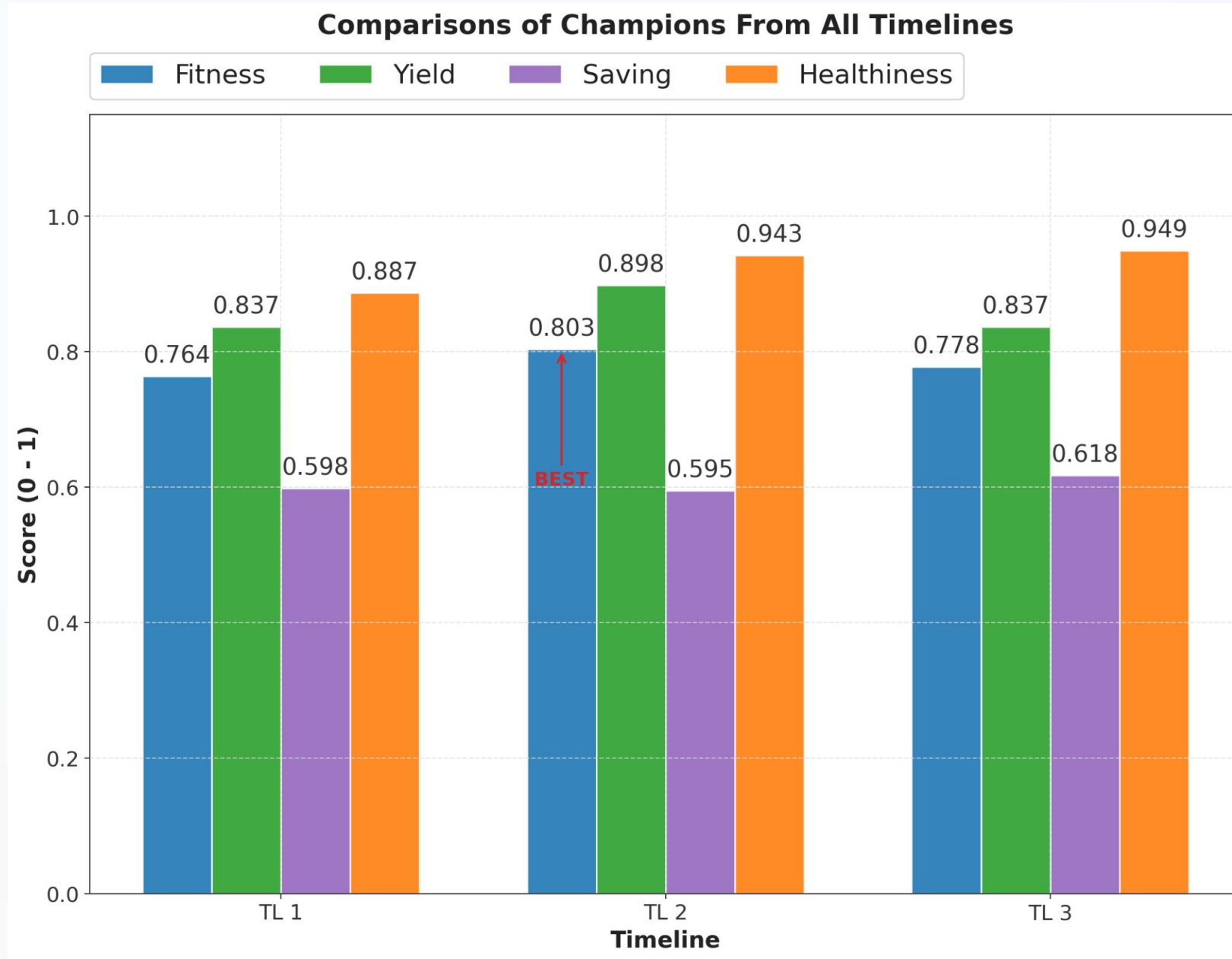
Data & Prompt (Continue)

```
_COUNTER_BLOCK = (  
    "Fixes: low_fullness->less food_interval|more food_qty. "  
    "low_oxygen->less O2_interval|more O2_dur. "  
    "low_immunity->less prob_interval|more prob_qty. "  
    "NH3/disease/parasite->move food/O2 away, reduce food_qty. "  
    "Over budget->increase O2_interval first, then food_interval. "  
    "Leftover food->pollution->NH3(30%). Start cheap, increase only if stat<0.5.\n"  
)
```

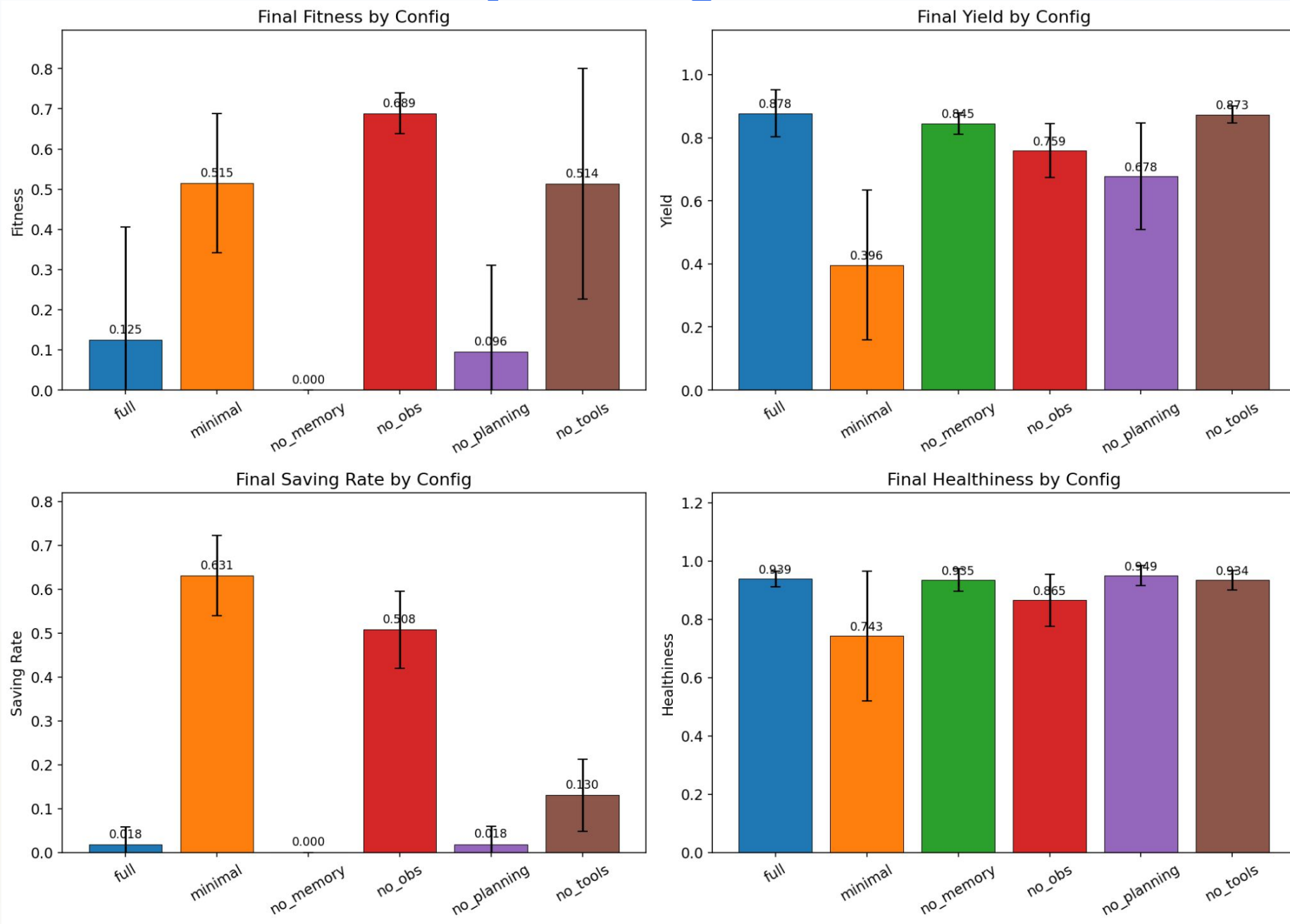
```
_FIRST_ROUND_BLOCK = (  
    "Day 0: Stock fish + set initial policy.\n"  
    "Call get_pond_info for max capacity (max_fish_count={max_fish}).\n"  
    "Yield=alive/max_fish_count. MORE fish stocked = higher possible yield.\n"  
    "Yield has the biggest weight (0.55). Try 40-50 fish if budget allows.\n"  
    "But more fish = more food needed = higher cost. Find the balance.\n"  
    "Call calculate_budget to verify. Output JSON with fish_count + 9 policy fields.\n"  
).format(max_fish=MAX_FISH_COUNT)
```

```
_ADJUST_BLOCK = (  
    "Read observation. If stat<0.5: fix it. If all OK: keep policy. "  
    "Output JSON (9 fields + early_quit). Only call budget tool if changing.\n"  
)
```

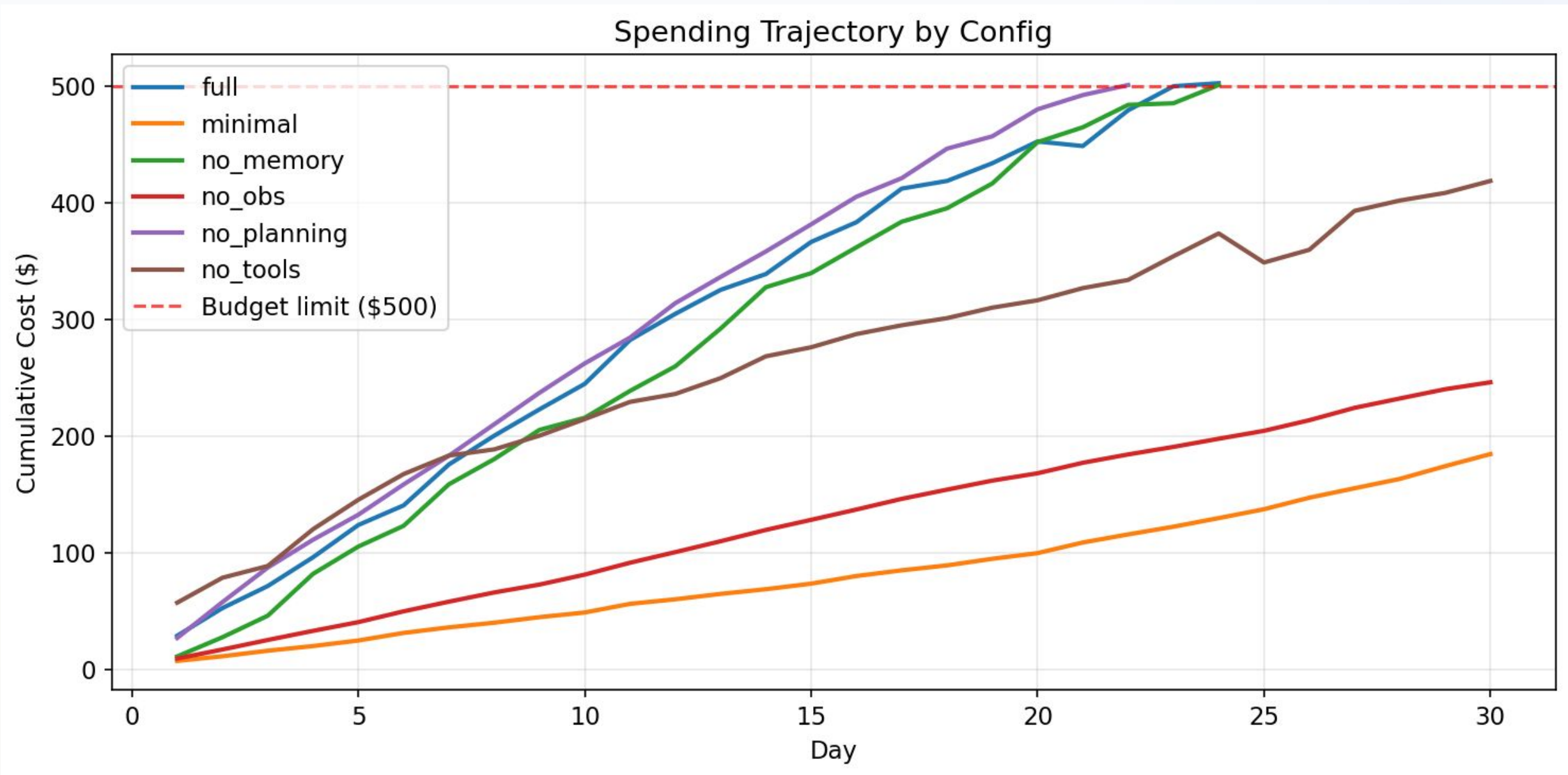
Results – Baseline Using Evolutionary Algorithm



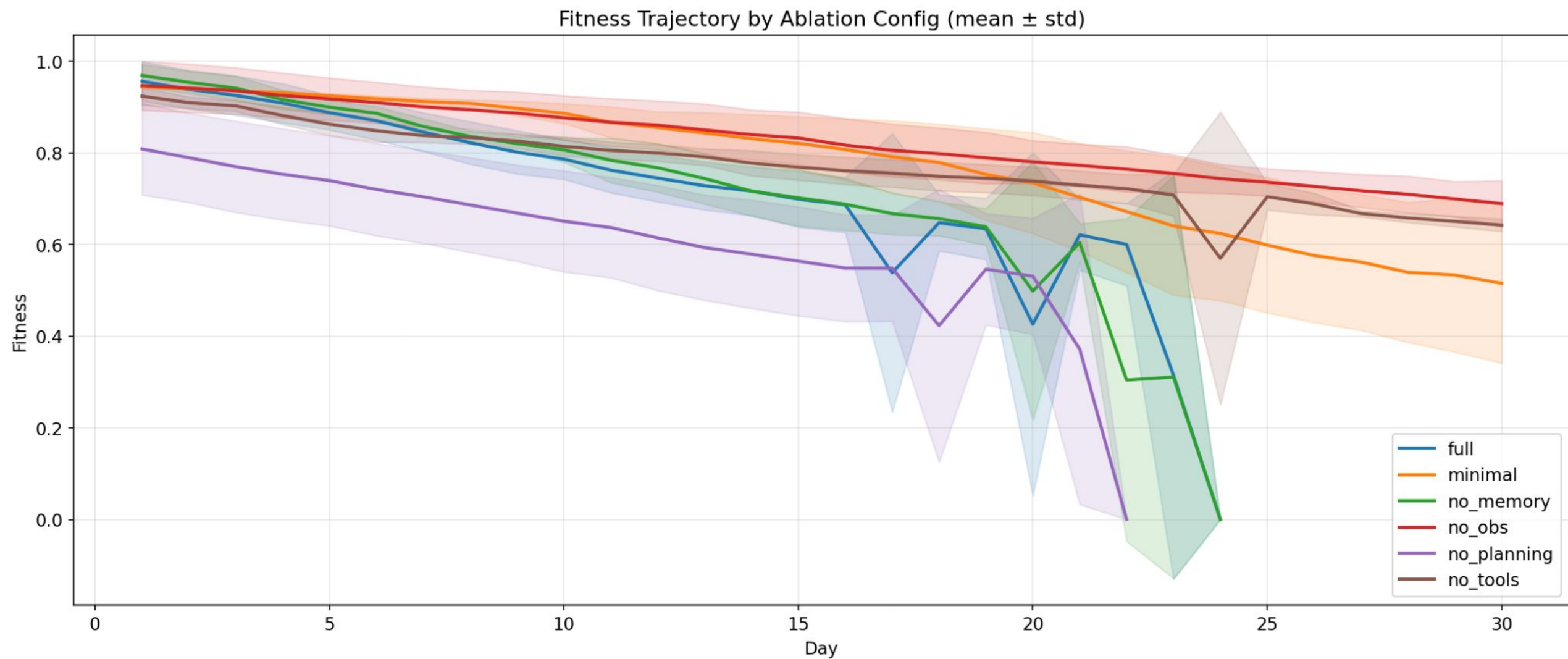
Results – Final Metrics by Configuration



Results - Cost by Configuration

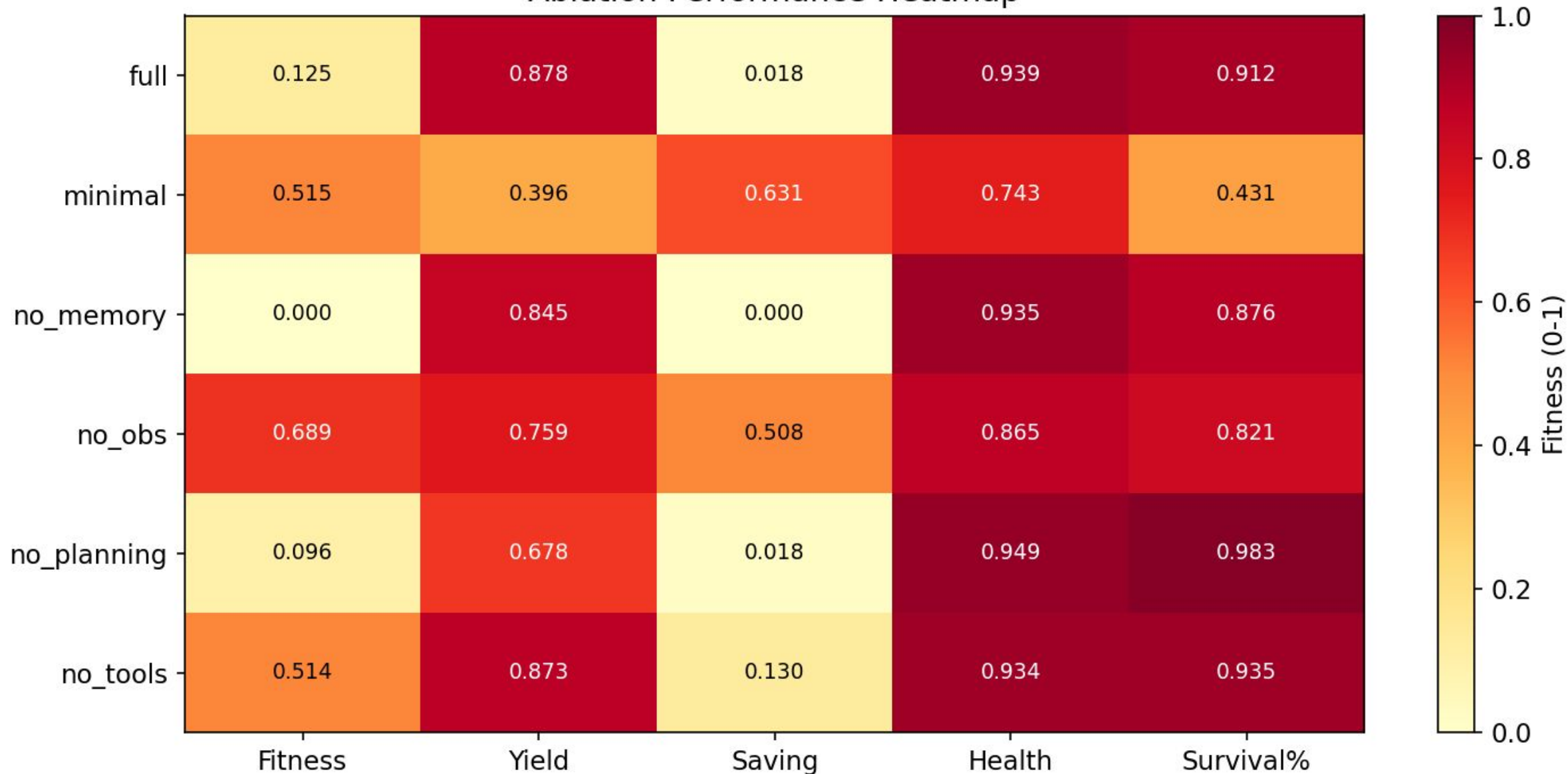


Results – Fitness Trajectory by Configuration



Results - Heatmap by Configuration

Ablation Performance Heatmap



Hallucination – 1

⚠ Issue: Inventing Metrics (all_logs.txt)

Proof:

Config: no_planning, Gen 2, Day 6:

>>> REFLECTION:...This will help save more of your fish's energy...

Explain:

Energy is not a tracked metric in the simulation

⚠ Issue: Inventing Metrics (all_logs.txt)

Proof:

Config: no_tools, Gen 4, Day 22:

>>> REFLECTION:...This will help you save more than 33% of your fish's fitness...

Explain:

Fitness is a final score, not a spendable resource pool

⚠ Issue: Hallucinating Units (trace_full.json)

Proof:

Gen 5, Attempt 2, Day 5: "The policy of reducing the food interval to 8 days, probiotic interval to 16 days..."

Explain:

The simulation intervals are strictly measured in hours, not days. This caused massive budget miscalculations.

Hallucination – 2

⚠ Issue: Modifying Locked Parameters (all_logs.txt)

Proof:

Config: full, Gen 1, Attempt 1 -> 2: Reason: Day 1 fitness $0.7725 < 0.9$. Had 30 fish (max 49). Reflection: Increase the fish_count to 40 by setting it directly since it's currently locked...

Explain:

The model explicitly acknowledges the parameter is locked, yet attempts to cheat the system by setting it anyway

⚠ Issue: Hallucinating Tool Arguments (trace_full.json)

Proof:

Gen 2, Attempt 2, Day 1: [TOOL_ERROR] calculate_budget: calculate_budget() got an unexpected keyword argument '**probiatic**_quantity'

Explain:

The model misspelled '**probiotic**' as 'probiatic', causing a tool crash

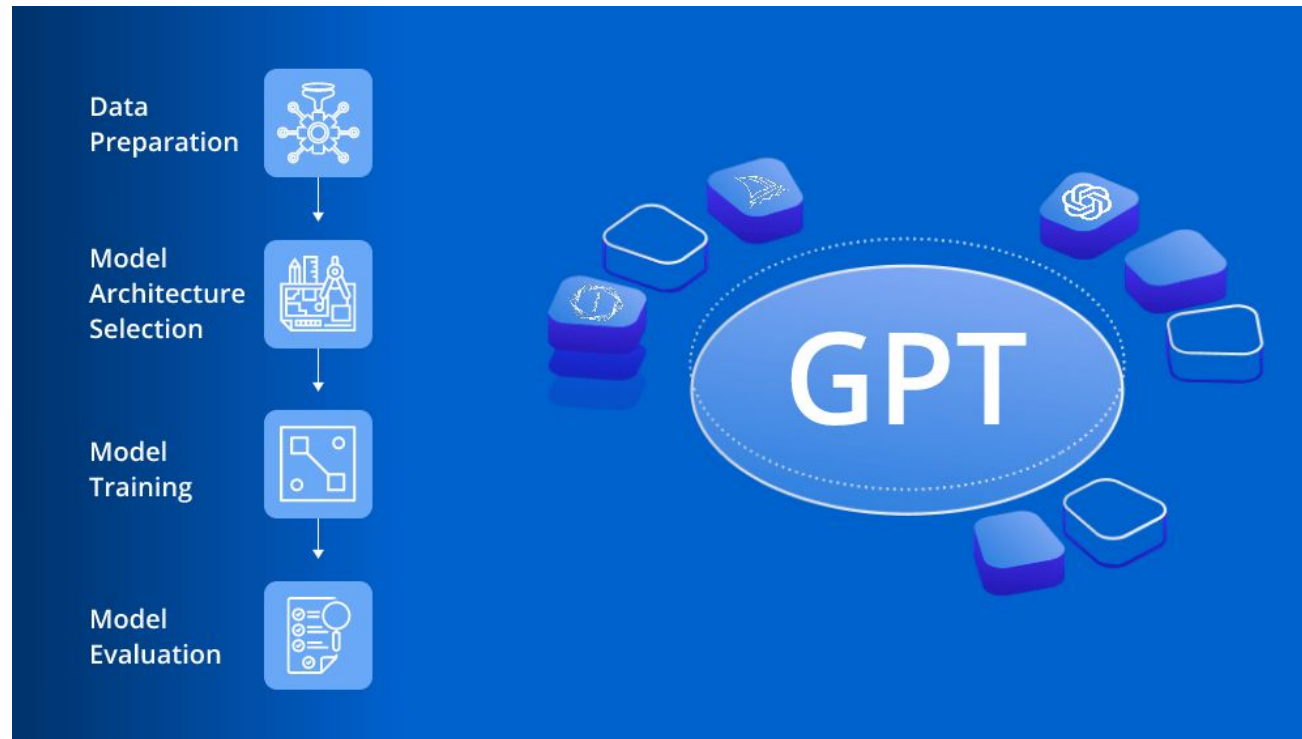
⚠ Issue: Hallucinating Units (trace_full.json)

Proof: Day 16 Reflection: >>> REFLECTION: D16(warn:153/144): To increase your fitness score, you might consider reducing the food_interval to a lower value (e.g., from 12h to 5h)... Change it to food_interval=5.

Day 17 Actual Execution: Day 17 |... | food=20x/12h@Random

Explain: The model explicitly decided to set the interval to 5 hours, but output 12 hours in the actual JSON policy, completely ignoring its own reasoning.

Next Steps & Future Work



Scale the Model

Test the full configuration on frontier models (Llama-3 70B, GPT-4o) to see if higher parameter can capture more constraint compared to the 7B model.

THANK YOU